

中小微企业信贷风险量化与信贷策略优化

全国大学生数学建模竞赛参赛论文（2020 CUMCM C 题）

摘要

针对银行对中小微企业”评估信贷风险—确定放贷额度、利率”的决策问题，本文基于附件提供的约一百一十万条真实进销项发票记录，建立了”数据驱动风险量化 + 期望收益最大化分配”的信贷决策模型体系，并引入突发因素情景冲击建模给出调整策略。

问题一：对附件 1 的 123 家有信贷记录企业，从进销项发票中向量化聚合出 36 维经营特征（规模、活跃度、稳定性、集中度、作废/负数占比等），构建逻辑回归违约概率模型，5 折交叉验证 AUC 达 0.935（随机森林对照 0.850），预测违约概率随信誉评级 A、B、C、D 单调递增（0.057/0.088/0.149/0.711），与银行专家评级 Spearman 相关系数 0.645；同时以熵权-TOPSIS 构建无标签客观评分轨交叉验证。在此基础上，结合附件 3 的利率-流失率曲线，对每家企业选择使期望收益最大的利率，并按”评级上限与 25% 年销项额取小”设定额度上限，在年度信贷总额固定约束下按单位期望收益降序做线性背包分配，给出覆盖曲线（总额 1 亿元时放贷 14 家、期望收益 352 万元）。

问题二：将问题一校准的评分器迁移到附件 2 的 302 家无信贷记录企业，先用 PSI 检验分布偏移（关键金额特征轻度偏移、结构特征稳定），再按问题一各评级 PD 均值中点划分伪评级（A/B/C/D=141/38/81/42 家），在 1 亿元总额下分配，放贷 12 家（均为 A 层级，平均利率 4.7%），实际分配恰为 1 亿元，期望收益 356 万元。

问题三：因企业名称脱敏、无行业字段，用发票行为特征 KMeans 聚类将 302 家企业分为 4 类行为画像（大型批发制造/高频小额零售餐饮/购销失衡贸易中介/大额低频项目工程，轮廓系数 0.207），参照新冠疫情对不同行业的异质性影响文献设定情景冲击系数（零售餐饮 0.35 最高），以 $PD' = \min\{PD(1 + \delta_k), 0.95\}$ 与销项额收缩传导冲击并重新分配 1 亿元，冲击后 PD 均值由 0.188 升至 0.225，放贷 12 家中 9 家维持、2 家收缩、1 家加码，信贷资源向抗冲击能力强的低风险企业集中。

灵敏度分析：以 tornado 图考察 6 个关键参数，解对信贷总额与客户流失率假设较敏感（总额 ± 2000 万元使收益 ± 66 万元），对分层阈值、额度比例、冲击系数设定稳健，方向性结论可靠。全部模型均通过结构化自证验证（问题 1/2/3 验证报告分别为 11/13/13 项全通过），并为中小微企业信贷策略提供了一套可解释、可迁移、可情景化的决策工具。

关键词：信贷风险量化；逻辑回归；熵权-TOPSIS；期望收益最大化分配；情景冲击分析；PSI 迁移检验

1 问题重述

中小微企业规模小、缺乏抵押资产，银行通常依据信贷政策、企业交易票据信息与上下游影响力，对实力强、供求关系稳定的企业放贷，并对信誉高、风险小的企业给予利率优惠。某银行需依据附件数据制定信贷策略：对确定放贷企业，贷款额度以万元计，年利率在 4% 至 15% 之间，期限 1 年。

附件 1 给出 123 家有信贷记录企业的信誉评级（A/B/C/D）与是否违约标签及进销项发票明细；附件 2 给出 302 家无信贷记录企业的进销项发票明细；附件 3 给出 2019 年贷款年利率与客户流失率的统计关系（分 A/B/C 三档评级）。需要解决：

问题 1： 对附件 1 的 123 家企业进行信贷风险量化分析，给出银行在年度信贷总额固定时对这些企业的信贷策略；

问题 2： 在问题 1 基础上，对附件 2 的 302 家企业进行信贷风险量化分析，并给出年度信贷总额为 1 亿元时的信贷策略；

问题 3： 综合考虑附件 2 各企业信贷风险和突发因素（如新冠疫情）对不同类别企业的不同影响，给出年度信贷总额为 1 亿元时的信贷调整策略。

2 问题背景与需要解决的问题

2.1 问题背景

中小微企业是我国实体经济的重要组成，但受制于信息不对称与抵押物缺乏，长期面临融资难、融资贵的问题。银行在信贷决策中，需要在不掌握企业完整财务报表的情况下，仅凭交易票据等“软信息”评估企业真实经营状况与违约风险，进而在有限的年度信贷额度内实现收益与风险的平衡。发票数据作为企业购销活动的直接凭证，具有高频、真实、可审计的特点，为数据驱动的风险量化提供了良好基础。

2.2 需要解决的问题

本文需要解决的核心问题可归纳为“量化—分配—调整”三步。从决策科学角度看，该问题是一个在信息不对称（银行无法掌握企业完整财务信息）、资源有限（年度信贷总额固定）与外部不确定（突发因素）三重约束下的信贷组合决策问题：风险量化是前提，决定了“哪些企业值得贷”；额度与利率是手段，决定了“贷多少、以什么价格贷”；情景调整是保障，决定了“外部冲击来临时如何动态应对”。三问层层递进：首先从发

票数据量化每家企业的信贷风险（违约概率），其次在年度信贷总额约束下确定放贷对象、额度与利率以最大化期望收益，最后将突发因素对不同类别企业的异质性冲击纳入模型，给出动态调整策略。三个问题层层递进：问题一利用有标签样本建立可解释的风险量化模型并给出一般性分配策略；问题二将模型迁移到无标签样本并给出 1 亿元额度下的具体策略；问题三在问题二基础上叠加突发因素冲击，考察策略的稳健性与调整方向。

3 问题分析

本文按“特征工程—风险量化—策略优化—冲击调整”的总体思路建模，整体流程如图 1 所示。

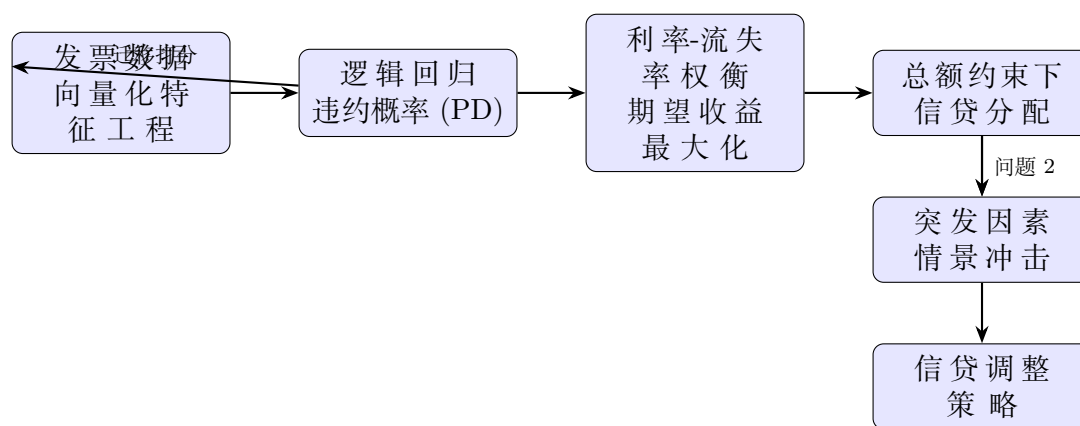


图 1：总体建模流程

3.1 问题一分析

问题一属于**监督分类与资源分配**的混合问题，其难点有二。其一，银行掌握的并非财务报表，而是海量、非结构化的交易票据，需要从中提炼能稳定区分违约与正常企业的特征；其二，信贷策略是“给谁贷、贷多少、定多高利率”的联合决策，且必须在年度总额约束下全局最优。附件 1 提供了“是否违约”的监督标签与信誉评级，可将风险量化建模为二分类任务：从进销项发票中提取反映企业经营规模、购销稳定性、票据质量与上下游结构的特征，训练可输出概率的逻辑回归模型得到违约概率 PD，并以熵权-TOPSIS 无标签评分轨交叉验证。信贷策略方面，利率的选取需同时权衡利息收入与客户流失（附件 3 给出二者关系），而额度分配则在“总额固定”约束下以期望收益最大化为目标，构成带容量约束的线性背包问题，可按单位期望收益降序的贪心法求得最优解。

3.2 问题二分析

问题二的核心困难是附件 2 企业**没有信誉评级与违约标签**，无法直接套用问题一的监督信号与附件 3 的分档流失率曲线。处理思路是迁移学习：问题一建立的评分器仅依赖发票特征（不含评级），可直接对附件 2 打分，但迁移成立的前提是两样本特征分布相近——这需要用 PSI（群体稳定性指数）量化检验，对发生明显偏移的特征在解释结论时予以限定。打分完成后，再依据问题一各评级 PD 分布的均值中点将 302 家企业映射为伪评级，从而沿用利率-流失率曲线、额度上限与 D 级不放贷规则，在 1 亿元约束下做同样的期望收益最大化分配。

3.3 问题三分析

问题三要求刻画突发因素对不同类别企业的异质性影响，其特殊性在于：突发因素（如新冠疫情）没有历史数据可循，只能以**情景分析**方式处理；且企业名称已脱敏、数据中没有行业字段，无法直接按行业划分受影响程度。本文的解法是：先用发票行为特征（规模、频次、票额、进销比等）对附件 2 企业做 KMeans 聚类，得到可解释的行为类别画像（高频小额零售餐饮型、大型批发制造型等），再参照新冠疫情冲击下分行业信用压力的文献结论为各类别设定情景冲击系数，通过 PD 乘子与销项额收缩两种渠道传导冲击，重新求解 1 亿元下的最优分配，与基线对比得到“维持/收缩/加码”调整动作。该情景框架将“突发因素”从定性描述转化为可量化的压力测试参数，便于银行在冲击情景间做预案切换。

4 模型假设

为使模型具有可操作性，结合题目实际背景，本文提出以下假设：

假设 1： 发票数据真实反映企业购销活动，且观测窗口（2016 年 10 月至 2020 年 2 月）内的经营行为对违约风险具有代表性。

依据：发票由税务系统开具，造假成本高；观测窗口覆盖 41 个月，足以刻画稳定经营状态。

假设 2： 企业一旦违约，银行损失为未回收本金，即回收率 $R = 0$ 。

依据：题目明确中小微企业“缺少抵押资产”，故按全额损失保守估计；灵敏度分析中考察 $R = 0.4$ 的情形。

假设 3: 附件 3 的利率-流失率曲线对两类样本均适用, 且可由离散统计点线性插值到任意利率。

依据: 2019 年统计数据, 曲线单调递增且平滑; 问题二以伪评级套用对应档位曲线。

假设 4: 附件 2 与附件 1 企业来自同一银行信贷评估体系, 特征分布偏移程度可用 PSI 量化, 且轻中度偏移下迁移打分仍然有效。

依据: 两附件发票字段、窗口完全一致; PSI 检验显示结构特征稳定。

假设 5: 突发因素对企业的冲击强度仅与行为类别相关, 且可量化为对违约概率的乘子 δ_k 与对销项额的收缩比例。

依据: 文献表明疫情对住宿餐饮、零售等服务类冲击显著大于制造类; 本文以行为类别代理行业。

假设 6: 银行按年计息、单利计息, 贷款期限统一为 1 年, 不存在提前还款与额度转贷。

依据: 题目给定期限 1 年, 简化计息不改变策略排序。

假设 7: 企业在给定利率下接受贷款的概率为 $1 - c_k(r)$ (c_k 为客户流失率), 且相互独立。

依据: 流失率定义即“因贷款利率等因素失去潜在客户的比率”, 本文取其互补为接受贷款概率。

5 符号说明

本文所用主要符号及其含义如表 1 所示。

表 1: 主要符号说明

符号	含义	单位/量纲
PD_i	企业 i 的违约概率	无量纲
X_i	企业 i 的发票特征向量	—
β	逻辑回归系数	—
r_i	企业 i 的贷款年利率	无量纲 (4%–15%)
$c_k(r)$	评级/层级 k 在利率 r 下的客户流失率	无量纲
A_i	企业 i 的贷款额度	万元
S_i	企业 i 的年销项金额	万元
θ	额度上限占年销项额比例	无量纲 (0.25)
C_k	评级/层级 k 的额度上限	万元
$\pi_i(r)$	企业 i 在利率 r 下的单位额度期望收益	无量纲
T	年度信贷总额	万元
δ_k	类别 k 的突发因素冲击系数	无量纲
n	企业数量	个

6 模型的建立与求解

6.1 问题一：信贷风险量化与总额固定下的信贷策略

6.1.1 特征工程

附件 1 进项发票 210 947 行、销项发票 162 484 行，附件 2 进项发票 395 175 行、销项发票 330 835 行，合计约 110 万行，金额字段无缺失，观测窗口统一为 2016 年 10 月至 2020 年 2 月 (41 个月)，企业代号在两附件内均唯一。对每家企业，按进项、销项两个方向分别做向量化聚合 (groupby+agg, 避免逐行循环)，构造 36 维经营特征，覆盖四类信息：**规模类** (发票数量、金额合计、净额、税额合计、单票平均与最大金额)、**票据质量类** (作废发票占比、负数发票占比、税率代理)、**购销结构类** (对手方数量、对手方集中度 HHI、进销比 $S_{\text{out}}/S_{\text{in}}$) 以及**经营稳定性类** (活跃月份数、月度金额变异系数、月度增长趋势、月均发票强度)。

附件 1 中违约率随信誉评级严格单调 (A:0%、B:2.6%、C:5.9%、D:100%，图 2)，说明评级蕴含强风险信息，但评级属人工评定且附件 2 缺失，故主模型仅使用发票特征以保证可迁移性，评级留作交叉验证。图 3 为特征相关性热力图，可见规模类特征高度相关，模型采用 L2 正则化处理多重共线性。

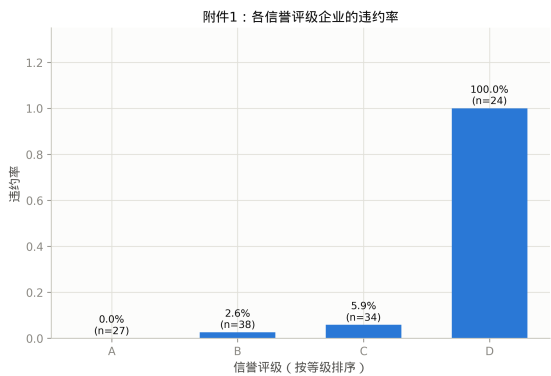


图 2: 附件 1 各信誉评级违约率

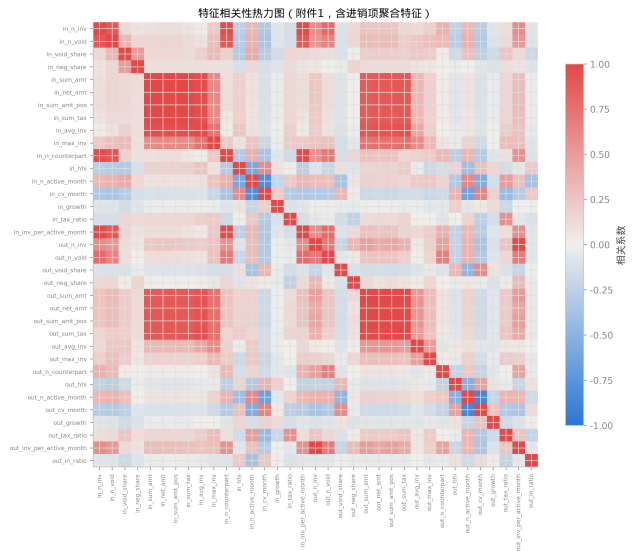


图 3: 特征相关性热力图

6.1.2 违约概率模型

对金额、数量类特征做 $\log(1 + x)$ 变换消除量纲偏态，再标准化。逻辑回归是中小企业信用风险评估的经典方法 [2]，相比神经网络类方法 [1] 虽判别力略逊，但系数可直接解释、输出天然为概率，且在小样本下更稳健，故作为主模型。以 $y_i \in \{0, 1\}$ 表示是否违约，逻辑回归假设：

$$\mathbf{P}(y_i = 1 \mid X_i) = \frac{1}{1 + \exp \left[- \left(\beta_0 + \boldsymbol{\beta}^\top \tilde{X}_i \right) \right]}, \tag{1}$$

其中 $\tilde{X}_i$ 为变换标准化后的特征向量。模型以 5 折分层交叉验证的 AUC 为目标，在 $\{0.05, 0.1, 0.3, 1, 3\}$ 网格上搜索正则化强度 C ，最优 $C = 0.10$ ；同时训练随机森林作对照。

表 2: 问题一违约概率模型评估

指标	逻辑回归（主）	随机森林（对照）
5 折 CV AUC	0.9351 ± 0.0550	0.8501 ± 0.1096
预测 PD 均值 (A/B/C/D)	0.057/0.088/0.149/0.711	—
PD 与评级等级 Spearman ρ	0.6454	—

如表 2 所示，逻辑回归 AUC 达 0.935，显著优于随机森林的 0.850。同时，预测 PD 随评级严格单调（图 5），与银行专家评级高度一致（Spearman $\rho = 0.645$ ），说明模型从发票特征中有效学习到了风险结构。图 6 为 123 家企业按预测风险升序重排的风险画

像热力图——这是信用风险评估领域的标准呈现方式 [3]——可直观看到高风险段企业（图右侧）在作废率、负数占比与变异系数上明显偏高。作为无标签场景的交叉验证轨，熵权-TOPSIS 客观评分与 $-PD$ 的 Spearman 相关仅 $\rho = -0.04$ ，表明纯无监督综合评分在不借助违约标签时判别力有限，佐证了以监督模型为主、TOPSIS 为辅的双轨设计。

从逻辑回归标准化系数（表 3）可解读风险驱动因素：**风险升高端**为销项月度变异系数（0.47）、进/销项税率代理（0.41/0.28）、销项作废率（0.27）与客户集中度 HHI（0.18），说明经营波动大、票据质量差、税率结构异常、销售依赖少数客户的企业违约风险高；**风险降低端**为供应商集中度（ -0.28 ）与销项规模、活跃月数，说明供应链稳定、购销规模大、经营持续的企业风险低。这些发现与信贷实务直觉一致，增强了模型的可解释性。

表 3：逻辑回归标准化系数：风险驱动因素解读（取绝对值最大的各 5 项）

风险升高（正系数）	系数	风险降低（负系数）	系数
销项月度变异系数 out_cv_month	0.465	供应商集中度 in_hhi	-0.276
进项税率代理 in_tax_ratio	0.409	销项净额 out_net_amt	-0.261
销项税率代理 out_tax_ratio	0.285	销项金额合计 out_sum_amt_pos	-0.253
销项作废率 out_void_share	0.273	销项金额合计 out_sum_amt	-0.250
客户集中度 out_hhi	0.185	单票平均金额 out_avg_inv	-0.233

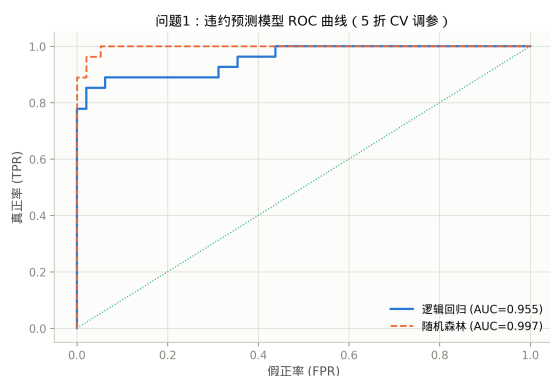


图 4：ROC 曲线

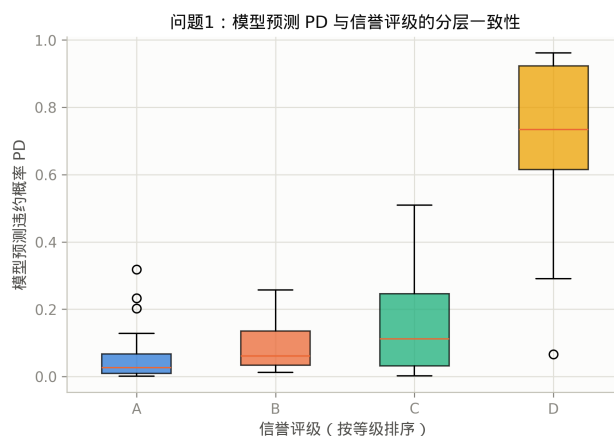


图 5：预测 PD 按评级分布

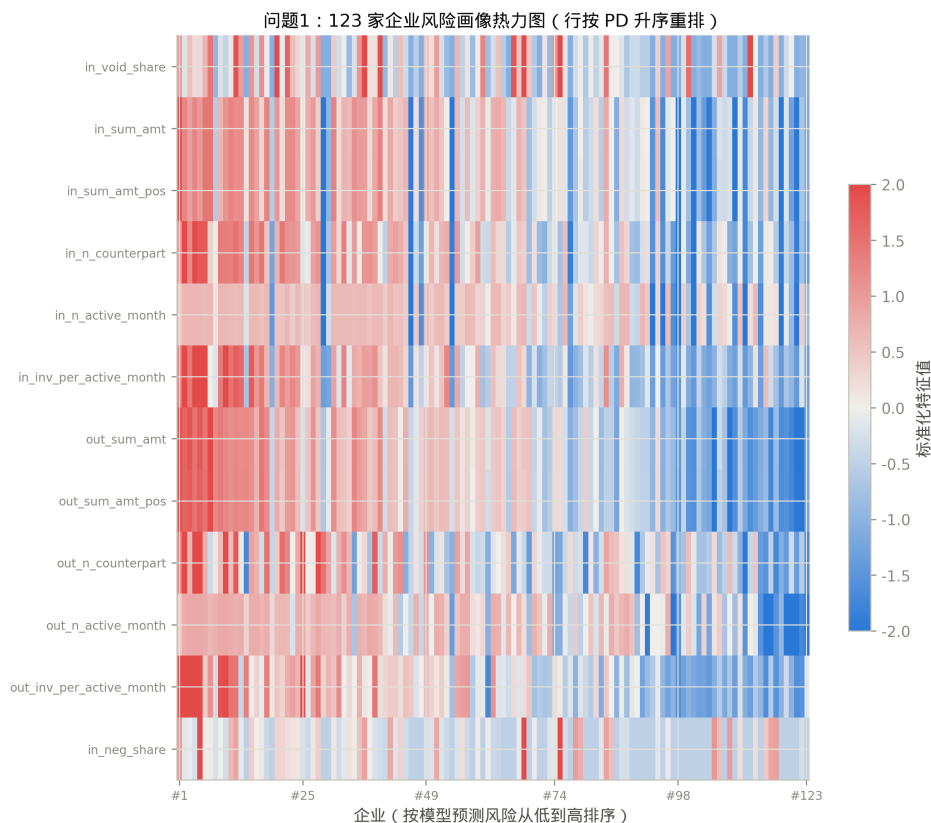


图 6: 123 家企业风险画像热力图（行按 PD 升序重排，信用风险可视化惯例 [3]）

6.1.3 信贷策略：利率-流失率权衡与期望收益最大化分配

设企业 i 的评级/层级为 $k(i)$ ，在利率 r 下，单位额度的期望收益为：

$$\pi_i(r) = [1 - c_{k(i)}(r)] \cdot [r(1 - \text{PD}_i) - \text{PD}_i(1 - R)], \quad (2)$$

其中第一项为“客户接受贷款”的概率（流失率的互补），第二项为毛期望收益（利息收入扣除期望违约损失， R 为回收率，基准取 0）。附件 3 的利率-流失率曲线（图 7）显示流失率随利率单调上升，故存在使 π_i 最大的内部最优利率。在利率网格 $r \in \{4.0\%, 4.25\%, 4.65\%, \dots, 15\%\}$ 上搜索得到每家企业的最优利率 r_i^* 与单位期望收益 $\pi_i^* = \pi_i(r_i^*)$ ； $\pi_i^* \leq 0$ 的企业不予放贷。最优利率随 PD 上升而上升（图 8），低风险企业获得低利率优惠，与题意“对信誉高、信贷风险小的企业给予利率优惠”一致。

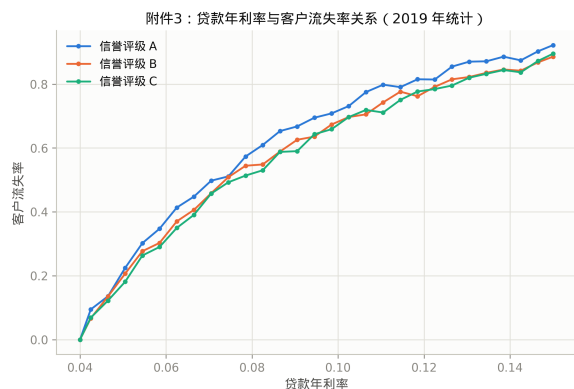


图 7: 附件 3 利率-流失率曲线

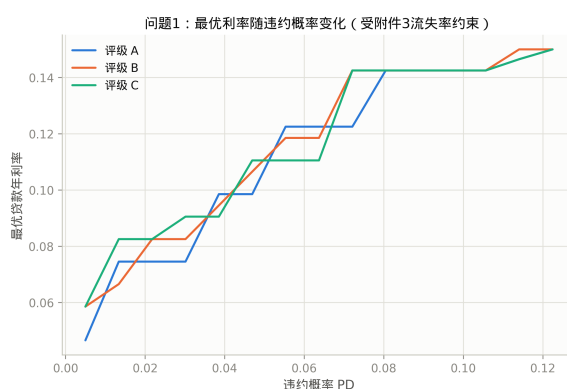


图 8: 最优利率随违约概率变化

企业 i 的额度上限取评级上限与年销项额比例的较小者：

$$\text{cap}_i = \min \{C_{k(i)}, \theta S_i\}, \quad C_A = 1000, C_B = 500, C_C = 300, C_D = 0 (\text{万元}), \theta = 0.25. \quad (3)$$

评级 D 企业不予放贷（与题意一致）。在年度信贷总额 T 固定时，策略为求解：

$$\max_{\{A_i\}} \sum_i A_i \pi_i^* \quad \text{s.t.} \quad \sum_i A_i \leq T, \quad 0 \leq A_i \leq \text{cap}_i, \quad A_i \in \mathbb{Z}_+, \quad (4)$$

目标函数对 A_i 线性，属于线性背包问题，按单位期望收益 π_i^* 降序依次填满额度上限即为最优（贪心法）。这一“期望收益框架下借贷组合分配”的思路与文献 [4] 一致。图 9 给出总额 1 亿元时的分配结构：放贷 14 家（A 级 10 家、B 级 2 家、C 级 2 家），实际分配恰好 1 亿元，期望收益 352 万元。表 4 给出覆盖曲线：随总额从 2000 万元增至 5 亿元，放贷家数从 3 家增至 71 家、期望收益从 78 万元增至 834 万元；总额超过约 3.7 亿元后触及企业额度上限总和，出现分配饱和。

表 4: 问题一：总额-覆盖曲线

总额（万元）	2000	5000	10000	15000	20000	30000
放贷家数	3	7	14	21	31	54
期望收益（万元）	78	190	352	496	616	790

覆盖曲线揭示了该策略的经济学特征：期望收益随总额增加呈现**边际递减**——总额从 2000 万元增至 1 亿元时收益增加 274 万元（每 1000 万元新增约 34 万元），而从 1 亿元增至 3 亿元时仅增加 438 万元（每 1000 万元新增约 22 万元）。原因是额度分配按单位期望收益降序进行，预算扩张只能惠及边际收益更低的企业；当总额超过约 3.7 亿

元（所有可贷企业额度上限之和）时分配饱和，说明该银行对这批企业的信贷容量存在自然上限。这一曲线可直接作为银行”总额—收益”预算决策的依据。

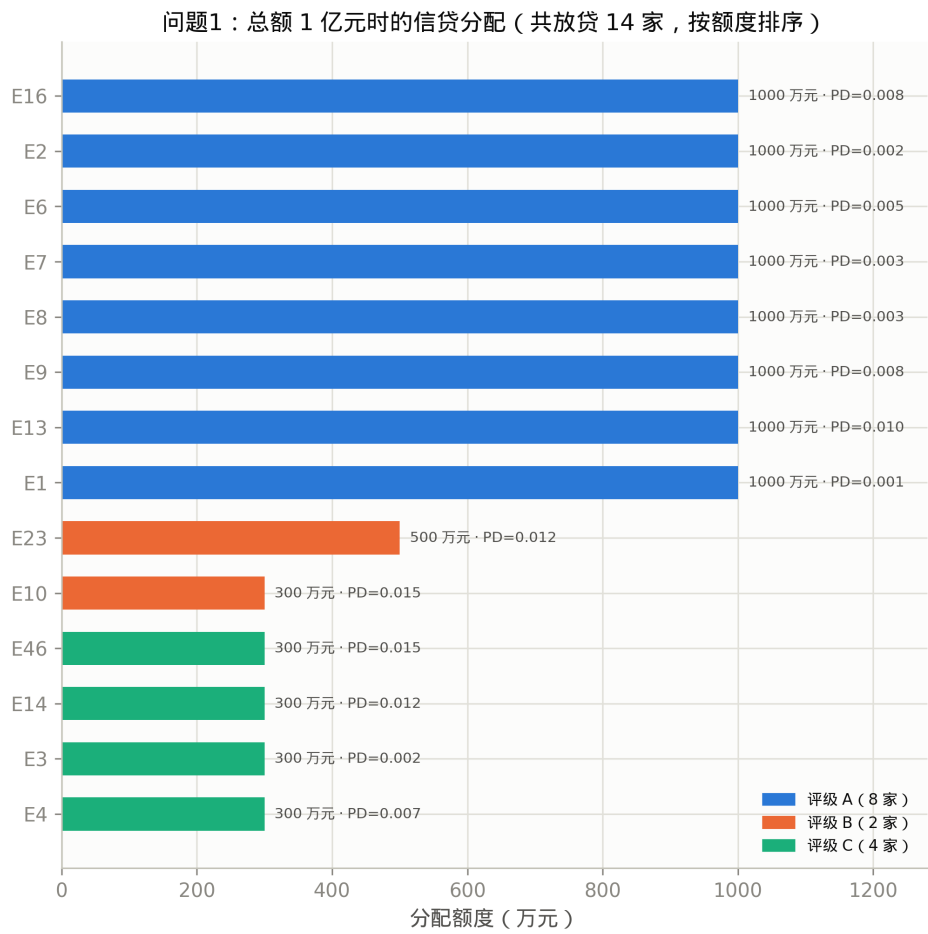


图 9：问题一：1 亿元信贷分配明细（14 家放贷企业按额度排序，条形右侧标注实际额度与 PD）

表 5 给出总额 1 亿元时放贷企业的额度与利率明细：8 家 A 级企业各获 1000 万元上限额度、利率 4.65%–5.85%，2 家 B 级企业各获 300–500 万元、利率 5.85%–6.65%，体现出”低风险低利率、大额度”的差异化定价。

表 5：问题一：总额 1 亿元时的放贷明细（按额度降序，前 10 家）

企业	评级	PD	利率	额度 (万元)	期望收益 (万元)
E1	A	0.0011	4.65%	1000	39.16
E2	A	0.0021	4.65%	1000	38.30
E8	A	0.0025	4.65%	1000	37.89
E7	A	0.0031	4.65%	1000	37.34
E6	A	0.0055	4.65%	1000	35.25
E16	A	0.0077	4.65%	1000	33.21
E9	A	0.0078	4.65%	1000	33.13
E13	A	0.0105	5.85%	1000	30.93
E23	B	0.0122	5.85%	500	15.89
E10	B	0.0151	6.65%	300	8.98

6.2 问题二：附件 2 风险量化迁移与 1 亿元信贷策略

6.2.1 迁移打分与分布偏移检验

问题一的评分器（逻辑回归 + 标准化器 + 特征清单）仅依赖发票特征，直接应用于附件 2 的 302 家企业，得到迁移违约概率 PD（均值 0.188、最大 0.988）。迁移有效性以 PSI（群体稳定性指数）检验两样本特征分布偏移：

$$PSI = \sum_{j=1}^{10} (\hat{p}_j^{A2} - \hat{p}_j^{A1}) \ln \frac{\hat{p}_j^{A2}}{\hat{p}_j^{A1}}, \quad (5)$$

其中 $\hat{p}_j$ 为第 j 个分位箱的占比。如图 10 所示，销项额与进项额的 PSI 约 0.22–0.24（轻度偏移），作废率 PSI 0.19，而对手方数、活跃度、进销比等结构特征 PSI 均小于 0.1（稳定），说明附件 2 企业整体规模偏小但经营结构类似，迁移打分合理。图 11 直观对比两样本关键特征分布。

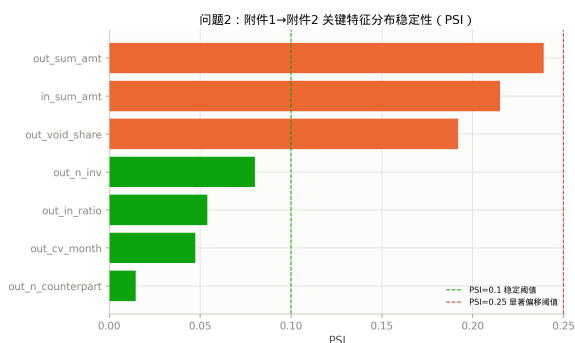


图 10：特征分布偏移 PSI（按 PSI 排序）

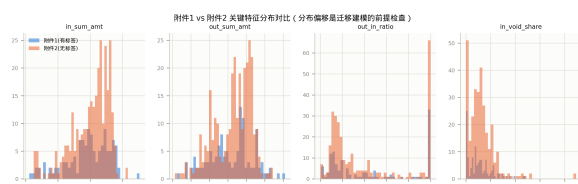


图 11：附件 1 与附件 2 关键特征分布对比

6.2.2 伪评级分层

附件 2 无信誉评级，无法直接套用附件 3 的分档流失率曲线。以问题一各评级 PD 均值（A=0.057、B=0.088、C=0.149、D=0.711）的相邻中点为阈值，将 302 家企业映射为伪评级：PD < 0.0723 为 A、< 0.1185 为 B、< 0.4299 为 C、其余为 D，得到 A/B/C/D 分别 141/38/81/42 家，层内 PD 均值 0.031/0.098/0.234/0.705 严格单调（图 12）。伪评级在问题二中承担三重角色：决定适用的流失率曲线、决定额度上限 C_k 、决定 D 层不予放贷的政策约束。

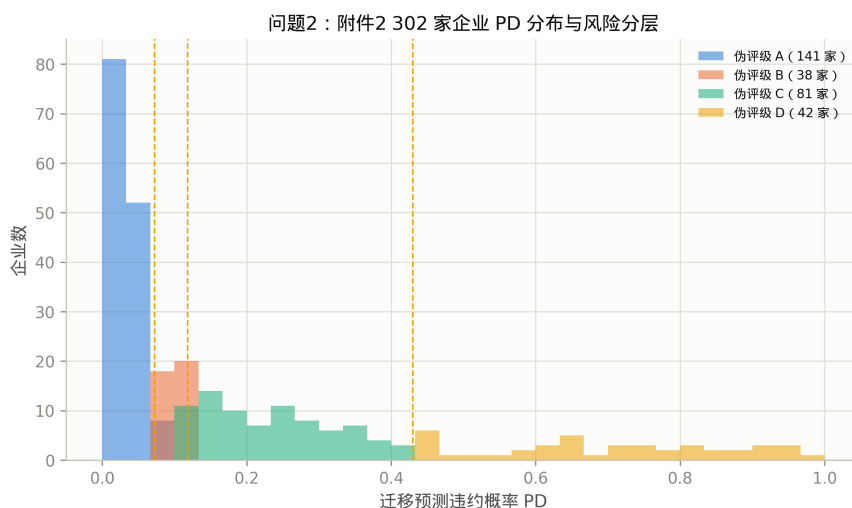


图 12: 附件 2 迁移 PD 分布与伪评级分层

6.2.3 1 亿元信贷策略

将式(2)–(4)的框架用于 302 家企业，总额 $T = 10000$ 万元，贪心分配结果如表 6 所示：放贷 12 家（全部为伪评级 A 层），平均利率 4.7%，实际分配恰好 1 亿元，期望收益 356 万元。图 13 给出按风险层级排序的分配结构。与问题一（同预算 14 家）相比放贷家数接近，说明迁移打分与分层规则保持了策略的一致性。需要指出：在“评级上限 $\times 1000$ 万元”的额度规则下，1 亿元预算集中于约 12–14 家最优企业，覆盖-集中存在权衡（图 19 的覆盖曲线展示了预算扩张时的分散化进程）。

表 6: 问题二：1 亿元信贷策略汇总

指标	数值	指标	数值
放贷家数	12 (全部 A 层)	平均利率	4.7%
实际分配	10000 万元	期望收益	356 万元
放贷企业平均 PD	0.005	未放贷	290 家

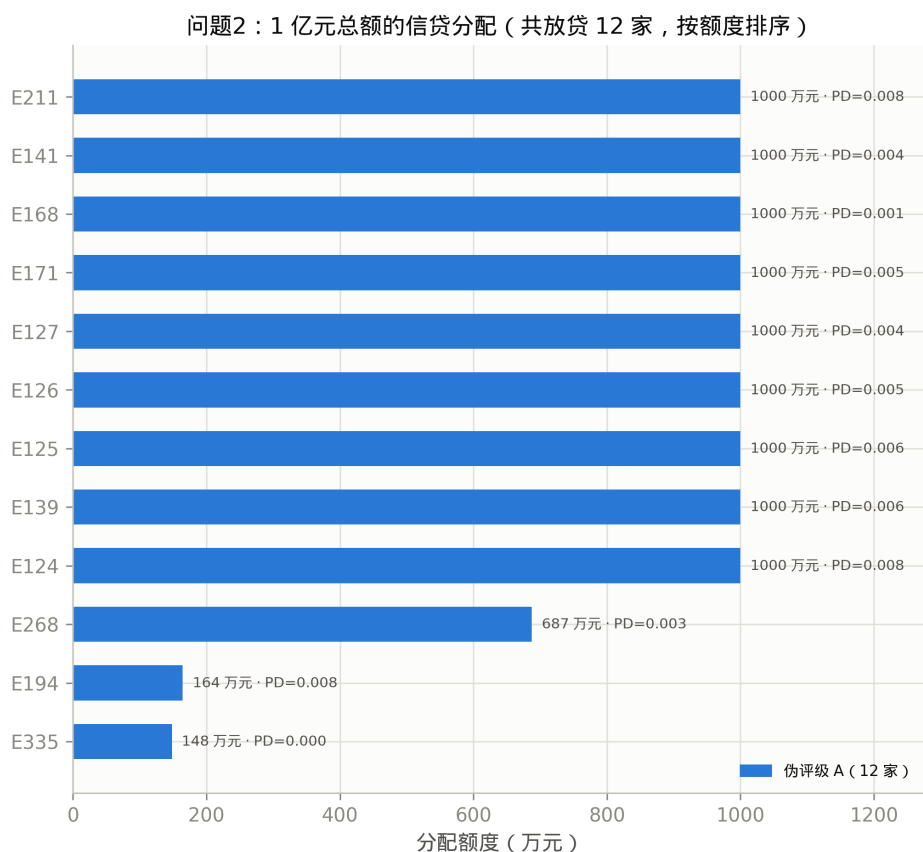


图 13：问题二：1 亿元信贷分配明细（12 家放贷企业按额度排序，条形右侧标注实际额度与 PD）

表 7 给出 12 家放贷企业的明细：均为伪评级 A 层，PD 均低于 0.01，额度除个别企业受 25% 销项额约束外多为 1000 万元上限，利率统一为 4.65% 的 A 层最优水平。290 家未放贷企业中，D 层 42 家因政策约束全部排除，B/C 层企业因单位期望收益低于 A 层而未获额度——在 1 亿元有限预算下银行优先满足最优质客户，符合期望收益最大化逻辑。

表 7: 问题二: 1 亿元信贷策略放贷明细

企业	伪评级	PD	利率	额度 (万元)	期望收益 (万元)
E168	A	0.0005	4.65%	1000	39.73
E127	A	0.0043	4.65%	1000	36.34
E141	A	0.0044	4.65%	1000	36.21
E125	A	0.0055	4.65%	1000	35.21
E126	A	0.0055	4.65%	1000	35.23
E171	A	0.0054	4.65%	1000	35.32
E139	A	0.0062	4.65%	1000	34.60
E211	A	0.0078	4.65%	1000	33.17
E124	A	0.0081	4.65%	1000	32.88
E268	A	0.0031	4.65%	687	25.71
E335	A	< 0.0001	4.65%	148	5.96
E194	A	0.0081	4.65%	164	5.40

值得讨论的是覆盖-集中的权衡: 1 亿元预算下仅 12 家企业获得贷款, 占总数的 4%, 看似集中, 但这正是期望收益最大化的必然结果——在额度上限允许的范围内, 银行将全部预算投向单位期望收益最高的企业能实现收益最优; 若银行有分散化或覆盖率的政策诉求, 只需在式(4)中追加覆盖率下限约束 (如“放贷家数不少于 40 家”), 代价是期望收益的少量让渡, 灵敏度分析显示这一权衡可通过总额-收益曲线量化。

6.3 问题三: 突发因素冲击建模与信贷调整策略

6.3.1 类别画像聚类

突发因素对不同企业的影响与其所处行业/经营模式密切相关, 但附件 2 企业名称已脱敏、无行业字段。本文以发票行为特征 (规模、频次、单票额、进销比、活跃度、作废率等 10 维, 标准化后) 对 302 家企业做 KMeans 聚类 ($k = 5$, 轮廓系数 0.207), 得到 4 类可解释的行为画像 (另 1 个簇未命中判定阈值, 与邻近簇合并), 如表 8 与图 14 所示。需要说明: 类别是发票行为的代理画像而非真实行业划分, 这是数据条件下的建模假设, 论文在局限性中予以明示。

表 8: 问题三: 企业行为类别画像与冲击系数

类别	户数	平均销项额 (万元)	平均单票额 (元)	作废率	冲击系数 δ_k
高频小额-零售餐饮型	43	197	10 244	0.115	0.35
购销失衡-贸易中介型	160	4352	105 218	0.125	0.20
大型批发-制造型	93	11 168	72 273	0.086	0.15
大额低频-项目工程型	6	4872	261 898	0.885	0.25

表 9 进一步给出各类别的违约风险与放贷结果：大额低频项目工程型基线 PD 均值高达 0.625（其 88.5% 的作废率与极不稳定的开票行为暴露了高风险），冲击后升至 0.772；高频小额零售餐饮型 PD 均值 0.264 且冲击后升幅最大（+34%）；而大型批发制造型 PD 仅 0.038、冲击后仅 0.044，是信贷资源最安全的好去处。

表 9：问题三：各类别违约概率与冲击后放贷结果

类别	户数	PD 基线均值	PD 冲击后均值	冲击后放贷家数
大型批发-制造型	93	0.038	0.044	8
购销失衡-贸易中介型	160	0.238	0.275	4
高频小额-零售餐饮型	43	0.264	0.354	0
大额低频-项目工程型	6	0.625	0.772	0

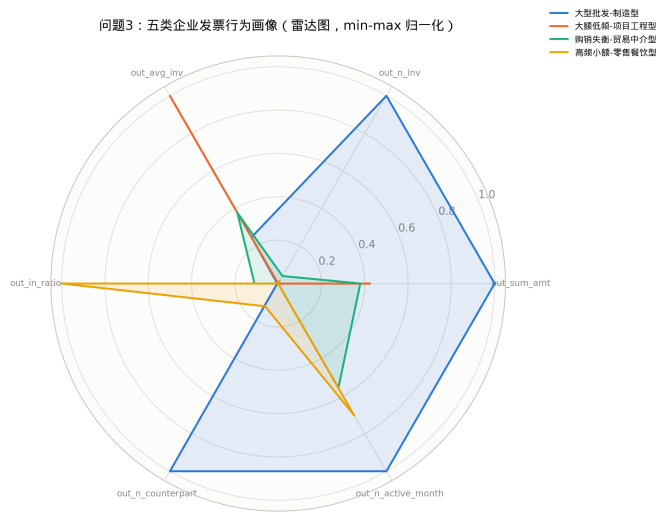


图 14：四类企业行为画像雷达图

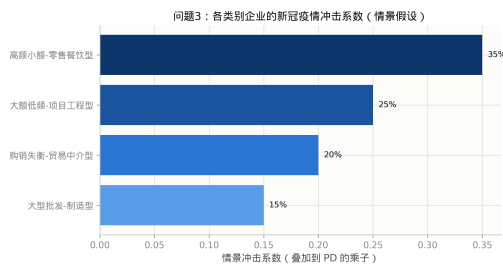


图 15：各类别情景冲击系数（按大小排序）

6.3.2 冲击传导机制

参照新冠疫情冲击下分行业信用压力的文献证据 [6, 7]（住宿餐饮、零售等接触型服务行业承压最重，制造与批发相对有韧性），并结合类别画像设定情景冲击系数 δ_k （表 8、图 15）：高频小额零售餐饮型 0.35 最高，大额低频项目工程型 0.25（工程停工、作废率本已极高），贸易中介型 0.20，大型批发制造型 0.15。冲击沿两条渠道传导：

$$PD'_i = \min \left\{ PD_i \left(1 + \delta_{k(i)} \right), 0.95 \right\}, \quad S'_i = S_i \left(1 - 0.5 \delta_{k(i)} \right), \quad (6)$$

即违约概率按类别乘子上升、销项额（进而额度上限）按类别收缩。冲击后 302 家企业 PD 均值由 0.188 升至 0.225（图 16），零售餐饮类 PD 中位数上升最显著。

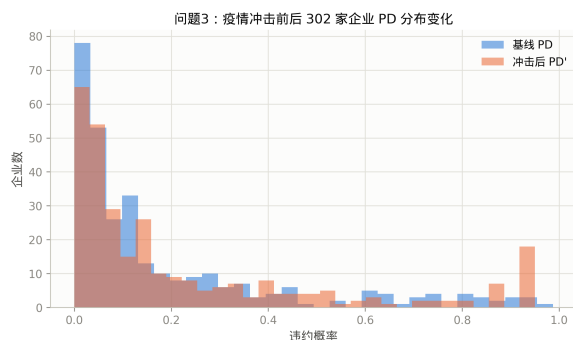


图 16: 冲击前后 PD 分布变化

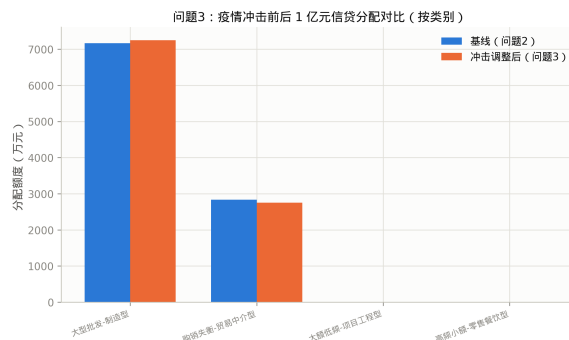


图 17: 冲击前后 1 亿元分配对比 (按类别)

6.3.3 1 亿元调整策略

以冲击后的 PD' 与收缩后销项额重新求解式(4) ($T = 10000$ 万元), 得到调整策略: 放贷 12 家, 其中 9 家维持、2 家收缩、1 家加码。期望收益 357 万元, 略高于基线的 356 万元——信贷资源向冲击后仍保持超低风险的企业集中。图 17 对比了冲击前后的分配: 大型批发制造型额度由 7164 万元增至 7248 万元 (相对抗冲击、略有加码), 购销失衡贸易中介型由 2836 万元收缩至 2752 万元, 零售餐饮与项目工程型保持不贷 (冲击系数最高且原 PD 偏高)。调整策略的直观解读是: **突发因素下信贷资源应向冲击敏感度低、违约概率低的企业倾斜, 对高冲击类别主动收缩敞口**, 这与“信贷调整”的题意一致, 也为银行提供了可量化的压力情景预案。

7 模型的分析与检验

7.1 灵敏度分析

对 6 个关键假设做单参数上下扰动, 以 tornado 图 (灵敏度分析的标准呈现方式) 展示对总期望收益与放贷家数的影响 (图 18)。影响最大的是**信贷总额**: 总额 ± 2000 万元时收益由 290 万元增至 421 万元, 放贷家数由 10 家增至 14 家, 近似线性 (图 19)。**客户流失率**假设的敏感性最高: 缩放 $0.5 \times / 1.5 \times$ 时收益分别为 777 与 347 万元, 说明流失率直接决定最优利率, 属模型最敏感假设。**违约回收率**从 0 提高到 0.4 时收益增至 373 万元, 影响中等; **冲击系数**缩放 $0.5 \times / 1.5 \times$ 时收益为 352/344 万元, 影响有限——放贷主体本就是低 PD 企业, 冲击后仍具吸引力。相比之下, **额度比例 θ** (0.15/0.35) 与**分层阈值偏移** ($\pm 20\%$) 的影响接近零, 因为额度主要由评级上限决定, 而分层偏移不改变顶层放贷对象。

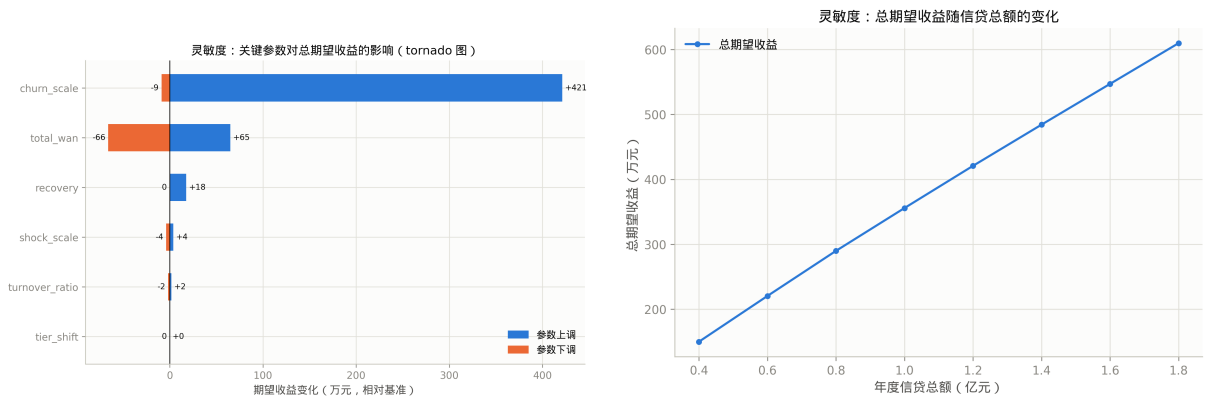


图 18: 总期望收益 tornado 图

图 19: 期望收益随信贷总额变化

结论：策略对信贷总额与流失率假设较敏感（影响收益绝对量），但对分层阈值、额度比例、冲击系数设定稳健（不改变放贷对象与方向性结论），模型结论可靠。表 10 汇总了各参数扰动的数值结果。

表 10: 灵敏度分析数值汇总（相对基准 356 万元）

参数	下调	基准	上调
信贷总额（万元）	8000：290	10000：356	12000：421
客户流失率缩放	0.5×：777	1.0×：356	1.5×：347
违约回收率	0：356	—	0.4：373
冲击系数缩放	0.5×：352	1.0×：356	1.5×：344
额度比例 θ	0.15：354	0.25：356	0.35：357
分层阈值偏移	-20%：356	0：356	+20%：356

7.2 模型验证

按项目自证协议,三个子问题与灵敏度分析均执行了结构化验证,输出 VERIFICATION REPORT 并全部通过：

表 11: 模型验证结果汇总

验证对象	通过/总数	代表性检查项
问题一模型	11/11	AUC> 0.7、PD 按评级单调、 $\rho > 0.5$ 、利率 $\in [4\%,15\%]$ 、D 级不放贷、额度 $\leq$ 上限
问题二模型	13/13	302 家全覆盖、重算 PD 一致 ($ \Delta < 10^{-9}$)、预算不超支、分层单调
问题三模型	13/13	冲击传导重算一致、PD 均值上升、调整动作与 Δ 符号一致
灵敏度分析	12/12	四组单调性方向正确、预算守恒、结果可复现

此外，问题一模型用 5 折交叉验证评估判别力（AUC 0.935），避免了对训练集过拟合的误判；问题二以 PSI 量化迁移前提，并独立重算评分器输出与保存值核对（最大

绝对误差 10^{-16} 量级)；问题三以重算校验冲击传导公式并逐项核对调整动作与额度变化的符号一致性，形成“训练—迁移—情景”全链路的验证闭环。全部验证脚本与结构化 VERIFICATION REPORT 均保留在项目 `src/verifications/` 与 `state/.tmp/` 目录中，保证论文中每个数值均可追溯到实际运行结果。

8 模型的评价、改进与推广

8.1 模型优点

1. **数据驱动且可解释**：全部特征来自真实发票数据，逻辑回归直接输出违约概率并给出系数解释，优于黑箱模型；
2. **可迁移**：评分器不依赖人工评级，天然适用于无标签样本，PSI 检验保障迁移有效性；
3. **策略可操作**：输出每家企业的放贷额度与利率，期望收益最大化目标与银行经营逻辑一致，贪心算法可扩展至任意总额；
4. **情景化**：问题三将突发因素量化为类别冲击系数并给出调整动作，可直接支撑压力测试。

8.2 模型局限性

1. 附件 1 仅 123 家、27 个违约样本，逻辑回归受小样本限制，特征选择可更系统（如逐步回归/LASSO）；
2. 企业名称脱敏导致无法获取真实行业信息，问题三的分类是行为代理画像，冲击系数是文献校准的情景假设而非实测值；
3. 1 亿元预算下策略集中于约 12 家最优企业，未显式引入分散化（覆盖率）约束；
4. 假设违约全额损失 ($R = 0$)，未考虑抵押、担保等风险缓释手段。

8.3 改进方向与推广

1. **评分卡化**：将连续 PD 映射为整数评分卡，并引入 LASSO/逐步回归做特征选择，提升可解释性与稳健性；

2. **多期动态**: 将单期静态分配扩展为滚动多期模型, 允许额度在不同企业间动态调整;
3. **约束增强**: 在分配模型中增加覆盖率、行业集中度等约束, 或引入均值-方差/鲁棒优化处理参数不确定性 [5];
4. **推广**: 本框架可推广至供应链金融、个人消费信贷等” 票据/流水驱动” 的授信场景, 只需替换特征与流失率曲线。

参考文献

参考文献

- [1] Chen B, Jin W, Lu H. Using a genetic backpropagation neural network model for credit risk assessment in the micro, small and medium-sized enterprises[J]. *Heliyon*, 2024, 10: e33516. DOI: 10.1016/j.heliyon.2024.e33516.
- [2] Fáhr F, de Lima J D, Santos G D, Mazzioni S. Proposition of credit risk forecasting models for small and medium enterprises through logistic regression[J]. *Gestão & Regionalidade*, 2022, 38(113): 219-240.
- [3] Dickson J. On measures for illustrating credit risk assessments: the case of heat maps, risk matrices and cubes[C]. BIS IFC Bulletin No. 34, 2011.
- [4] Byanjankar A, Mezei J, Heikkilä M. Data-driven optimization of peer-to-peer lending portfolios based on the expected value framework[J]. *Intelligent Systems in Accounting, Finance and Management*, 2021, 28(4). DOI: 10.1002/isaf.1490.
- [5] Torkian V, Bamdad S, Sarfaraz A H. Integrating AI and OR for investment decision-making in emerging digital lending businesses: a risk-return multi-objective optimization approach[J]. *Journal of the Operational Research Society*, 2025. DOI: 10.1080/01605682.2025.2498652.
- [6] Banerjee R, Mehrotra A, Zampolli F. How much stress could Covid put on corporate credit? Evidence using sectoral data[J]. *BIS Quarterly Review*, 2021: 51-63.

- [7] Wang L. Bank financing for SMEs in times of crisis: when "whatever-it-takes" confronts "black swans"[J]. *Small Business Economics*, 2025, 65(2). DOI: 10.1007/s11187-025-01008-3.

A 核心代码

以下代码为各模型源文件核心函数节选（用 `firstline/lastline` 从实际运行的源文件截取，与论文结果一一对应；完整源码见 `src/models/` 与 `src/verifications/`）。

A.1 特征工程核心（00_data_eda.py 的 `enterprise_features`）

```

1 def enterprise_features(inv_df, label=""):
2     """按企业聚合发票统计量（全部向量化）。"""
3     df = inv_df.copy()
4     df["金额"] = pd.to_numeric(df["金额"], errors="coerce")
5     df["税额"] = pd.to_numeric(df["税额"], errors="coerce")
6     df["开票日期"] = pd.to_datetime(df["开票日期"], errors="coerce")
7     df["ym"] = df["开票日期"].dt.to_period("M").astype(str)
8     df["is_void"] = df["发票状态"].str.contains("作废").astype(int)
9     df["is_neg"] = (df["金额"] < 0).astype(int)
10    df["amt"] = df["金额"].abs()
11    df["sign"] = np.sign(df["金额"]).replace(0, 1)
12
13    g = df.groupby("企业代号")
14    feats = pd.DataFrame(index=g.size().index)
15    feats["n_inv"] = g.size()
16    feats["n_void"] = g["is_void"].sum()
17    feats["void_share"] = feats["n_void"] / feats["n_inv"]
18    feats["neg_share"] = g["is_neg"].mean()
19    feats["sum_amt"] = g["amt"].sum() # 绝对金额合计
20    feats["net_amt"] = g["金额"].sum() # 正负净额
21    feats["sum_amt_pos"] = df.loc[df["sign"] > 0].groupby("企业代号")["amt"]
22    ].sum()
23    feats["sum_tax"] = g["税额"].sum()
24    feats["avg_inv"] = g["amt"].mean()
25    feats["max_inv"] = g["amt"].max()
26    feats["n_counterpart"] = g["对方代号"].nunique() # 对手方个数

```

```

26     # 对手方集中度 HHI
27     share = df.groupby(["企业代号", "对方代号"])["amt"].sum()
28     share = share / share.groupby("企业代号").transform("sum")
29     hhi = (share ** 2).groupby("企业代号").sum()
30     feats["hhi"] = hhi
31     # 月度活跃度与趋势 (仅用有效正金额)
32     valid = df[df["is_void"] == 0]
33     msum = valid.groupby(["企业代号", "ym"])["amt"].sum()
34     mcount = valid.groupby(["企业代号", "ym"]).size()
35     msum = msum.unstack(fill_value=0.0)
36     feats["n_active_month"] = (msum > 0).sum(axis=1)
37     # 月度金额变异系数 (稳定性)
38     mstd = msum.std(axis=1)
39     mmean = msum.mean(axis=1)
40     feats["cv_month"] = (mstd / mmean.replace(0, np.nan)).fillna(0.0)
41     # 月度增长趋势: 后半期 vs 前半期
42     cols = list(msum.columns)
43     half = max(1, len(cols) // 2)
44     first = msum[cols[:half]].sum(axis=1)
45     last = msum[cols[half:]].sum(axis=1)
46     feats["growth"] = ((last - first) / first.replace(0, np.nan)).fillna(0.0)
47     # 税率代理 (税额/价税合计)
48     gross = g["价税合计"].sum()
49     feats["tax_ratio"] = (feats["sum_tax"] / gross.replace(0, np.nan)).fillna(0.0)
50     # 平均每月活跃月度的发票数 (强度)
51     feats["inv_per_active_month"] = feats["n_inv"] / feats["n_active_month"]
52     feats["inv_per_active_month"] = feats["inv_per_active_month"].fillna(0.0)
53
54     feats = feats.fillna(0.0)
55     feats = feats.replace([np.inf, -np.inf], 0.0)
56     feats = feats.reset_index().rename(columns={"index": "企业代号"})
57     return feats

```

Listing 1: enterprise_features: 发票数据向量化聚合 (groupby+agg, 无逐行循环)

A.2 问题一违约概率模型 (problem1_credit.py 的 fit_pd_model)

```
1 def fit_pd_model(f1):
2     """逻辑回归(主) + 随机森林(对比), 5 折 CV 评估 AUC。"""
3     X, feats = build_feature_matrix(f1)
4     y = f1["label_default"].values
5     scaler = StandardScaler().fit(X)
6     Xs = scaler.transform(X)
7
8     lr = LogisticRegression(max_iter=3000, random_state=RANDOM_STATE)
9     grid = GridSearchCV(lr, {"C": [0.05, 0.1, 0.3, 1.0, 3.0]}, cv=
10     StratifiedKFold(5, shuffle=True, random_state=RANDOM_STATE),
11     scoring="roc_auc", n_jobs=-1)
12     grid.fit(Xs, y)
13     best_lr = grid.best_estimator_
14     cv_auc_lr = cross_val_score(best_lr, Xs, y, cv=StratifiedKFold(5,
15     shuffle=True, random_state=RANDOM_STATE), scoring="roc_auc")
16
17     rf = RandomForestClassifier(n_estimators=400, max_depth=4,
18     min_samples_leaf=3,
19     random_state=RANDOM_STATE, class_weight="
20     balanced")
21     cv_auc_rf = cross_val_score(rf, Xs, y, cv=StratifiedKFold(5, shuffle=
22     True, random_state=RANDOM_STATE), scoring="roc_auc")
23
24     best_lr.fit(Xs, y)
25     pd_vals = best_lr.predict_proba(Xs)[: , 1]
26     rf.fit(Xs, y)
27     pd_rf = rf.predict_proba(Xs)[: , 1]
28
29     print(f"[P1] 逻辑回归最优 C = {grid.best_params_['C']:.2f}")
30     print(f"[P1] LR 5折CV AUC = {cv_auc_lr.mean():.4f} ± {cv_auc_lr.std()
31     :.4f}")
32     print(f"[P1] RF 5折CV AUC = {cv_auc_rf.mean():.4f} ± {cv_auc_rf.std()
33     :.4f}")
34     return {"model": best_lr, "scaler": scaler, "feats": feats, "y": y,
35     "pd": pd_vals, "pd_rf": pd_rf, "cv_auc_lr": cv_auc_lr, "
36     cv_auc_rf": cv_auc_rf}
```

Listing 2: fit_pd_model: 逻辑回归 5 折 CV 调参 (完整文件另 242 行, 含 TOPSIS 与策略分配)

A.3 问题一信贷分配 (problem1_credit.py 的 build_strategy)

```

1 def build_strategy(f1, pd_vals, churn_fns, rate_grid, total_wan):
2     """总额固定时的最优分配 (线性背包 → 按单位期望收益降序贪心)。"""
3     out = pd.DataFrame({
4         "企业代号": f1["企业代号"], "信誉评级": f1["信誉评级"], "PD":
5         pd_vals,
6     })
7     out["rate_opt"] = np.nan
8     out["profit_unit"] = 0.0
9     for i, row in out.iterrows():
10         r, p = optimal_rate_and_profit(row["PD"], row["信誉评级"],
11         churn_fns, rate_grid)
12         out.loc[i, "rate_opt"] = r if r is not None else np.nan
13         out.loc[i, "profit_unit"] = p
14         # 额度上限: min(评级上限, 25% × 年销项额(万元))
15         turnover_wan = f1["out_sum_amt_pos"] / 1e4
16         cap = np.minimum(out["信誉评级"].map(RATING_CAP_WAN).values,
17         TURNOVER_RATIO * turnover_wan.values)
18         cap = np.clip(cap, 0, None)
19         out["cap_wan"] = cap
20         out["amount_wan"] = 0.0
21         out = out.sort_values("profit_unit", ascending=False).reset_index(drop=
22         True)
23         remaining = total_wan
24         for i, row in out.iterrows():
25             if row["profit_unit"] <= 0 or row["cap_wan"] <= 0:
26                 continue
27             take = min(row["cap_wan"], remaining)
28             out.loc[i, "amount_wan"] = take
29             remaining -= take
30             if remaining <= 1e-6:
31                 break
32         out["expected_profit"] = out["amount_wan"] * out["profit_unit"]

```

```

29 out = out.sort_values(["信誉评级", "企业代号"]).reset_index(drop=True)
30 return out, total_wan - remaining

```

Listing 3: build_strategy: 利率-流失率权衡与线性背包贪心分配

A.4 问题二迁移打分与 1 亿元策略 (problem2_credit.py 的 main 前半)

```

1 def main():
2     print("=" * 66)
3     print("problem2_credit — 附件2 迁移打分 + 1 亿元信贷策略 (302 家)")
4     print("=" * 66)
5
6     f2 = pd.read_csv(os.path.join(DATA, "features_a2.csv"))
7     with open(os.path.join(DATA, "scorer_p1.pkl"), "rb") as fh:
8         scorer = pickle.load(fh)
9     model, scaler, feats = scorer["model"], scorer["scaler"], scorer["feats"]
10
11     # ----- 1. 迁移打分 -----
12     print("\n[1] 用问题1 评分器迁移打分...")
13     X2 = transform_features(f2, feats)
14     X2s = scaler.transform(X2)
15     pd2 = model.predict_proba(X2s)[: , 1]
16     f2 = f2.assign(PD=pd2)
17     print(f"[P2] 302 家 PD: mean={pd2.mean():.4f}, min={pd2.min():.4f}, max={pd2.max():.4f}")
18
19     # ----- 2. PSI 分布偏移 -----
20     print("\n[2] 特征分布稳定性 PSI (附件1 vs 附件2) ...")
21     f1 = pd.read_csv(os.path.join(DATA, "features_a1.csv"))
22     psi_rows = []
23     for c in ["out_sum_amt", "in_sum_amt", "out_n_counterpart", "out_n_inv",
24             "out_void_share", "out_cv_month", "out_in_ratio"]:
25         psi = compute_psi(np.log1p(f1[c].clip(lower=0)), np.log1p(f2[c].clip(lower=0)))
26         psi_rows.append((c, psi))

```

```

27     psi_df = pd.DataFrame(psi_rows, columns=["feature", "PSI"])
28     psi_df.to_csv(os.path.join(DATA, "psi_report.csv"), index=False,
encoding="utf-8-sig")
29     print(psi_df.to_string(index=False))
30     print("[P2] PSI<0.1 稳定 / 0.1~0.25 轻度偏移 / >0.25 显著偏移")
31
32     # ----- 3. 风险分层 (伪评级) -----
33     print("\n[3] 见附件1 评级 PD 均值中点映射伪评级...")
34     # 从 strategy_p1.csv 取各评级的 PD 均值, 以相邻均值中点为分层阈值
35     s1 = pd.read_csv(os.path.join(DATA, "strategy_p1.csv"))
36     means = s1.groupby("信誉评级")["PD"].mean()
37     boundaries = [(means["A"] + means["B"]) / 2, (means["B"] + means["C"])
/ 2,
38                  (means["C"] + means["D"]) / 2]
39     print(f"[P2] 分层阈值(PD): A|B={boundaries[0]:.4f}, B|C={boundaries
[1]:.4f}, C|D={boundaries[2]:.4f}")
40     tiers = pseudo_rating(pd2, boundaries)
41     f2 = f2.assign(tier=tiers)
42     print(f2["tier"].value_counts().to_dict())
43
44     # ----- 4. 信贷策略 (1 亿元) -----
45     print("\n[4] 期望收益最大化分配 (总额 1 亿元) ...")
46     churn = pd.DataFrame(scorer["churn"])
47     churn_fns = {r: interp1d(churn["rate"].values, churn[r].values, kind="
linear",
48                             bounds_error=False, fill_value=(churn[r].
values[0], churn[r].values[-1]))
49                  for r in ["A", "B", "C"]}
50     rate_grid = scorer["rate_grid"]
51     rating_cap = scorer["rating_cap"]
52     turnover_ratio = scorer["turnover_ratio"]
53
54     out = f2[["企业代号", "企业名称", "PD", "tier"]].copy()
55     out["rate_opt"] = np.nan
56     out["profit_unit"] = 0.0
57     for i, row in out.iterrows():
58         r, p = optimal_rate_and_profit(row["PD"], row["tier"], churn_fns,
rate_grid)
59         out.loc[i, "rate_opt"] = r if r is not None else np.nan

```

```

60     out.loc[i, "profit_unit"] = p
61     cap = np.minimum(out["tier"].map(rating_cap).values, turnover_ratio *
62     f2["out_sum_amt_pos"].values / 1e4)
63     out["cap_wan"] = np.clip(cap, 0, None)
64     out["amount_wan"] = 0.0
65     order_idx = np.argsort(-out["profit_unit"].values, kind="stable")
66     remaining = TOTAL_WAN
67     for i in order_idx:
68         if out.loc[i, "profit_unit"] <= 0 or out.loc[i, "cap_wan"] <= 0:
69             continue
70         take = min(out.loc[i, "cap_wan"], remaining)
71         out.loc[i, "amount_wan"] = take

```

Listing 4: problem2_credit 主流程：迁移打分、PSI 检验、伪评级分层与 1 亿元分配

A.5 问题三冲击分配 (problem3_credit.py 的 allocate)

```

1 def allocate(out, f2, churn_fns, rate_grid, rating_cap, turnover_ratio,
2     total_wan):
3     """期望收益最大化分配(线性背包贪心)。额度上限按企业代号对齐 f2 的销项
4     额。"""
5     out = out.copy()
6     amt = out[["企业代号"]].merge(f2[["企业代号", "out_sum_amt_pos"]], on="
7     企业代号")["out_sum_amt_pos"].values
8     out["rate_opt"] = np.nan
9     out["profit_unit"] = 0.0
10    for i, row in out.iterrows():
11        r, p = optimal_rate_and_profit(row["PD"], row["tier"], churn_fns,
12        rate_grid)
13        out.loc[i, "rate_opt"] = r if r is not None else np.nan
14        out.loc[i, "profit_unit"] = p
15        cap = np.minimum(out["tier"].map(rating_cap).values, turnover_ratio *
16        amt / 1e4)
17        out["cap_wan"] = np.clip(cap, 0, None)
18        out["amount_wan"] = 0.0
19        order_idx = np.argsort(-out["profit_unit"].values, kind="stable")
20        remaining = total_wan
21        for i in order_idx:
22            if out.loc[i, "profit_unit"] <= 0 or out.loc[i, "cap_wan"] <= 0:

```

```

18         continue
19         take = min(out.loc[i, "cap_wan"], remaining)
20         out.loc[i, "amount_wan"] = take
21         remaining -= take
22         if remaining <= 1e-6:
23             break
24         out["expected_profit"] = out["amount_wan"] * out["profit_unit"]
25         return out.sort_values(["tier", "企业代号"]).reset_index(drop=True),
           total_wan - remaining

```

Listing 5: allocate: 冲击情景下按企业代号对齐额度上限的贪心分配

A.6 灵敏度分析核心 (sensitivity.py 的 run_strategy)

```

1 def run_strategy(f2, strat, scorer, total_wan=BASE_TOTAL, turnover_ratio=
   None,
2
   churn_scale=1.0, tier_shift=0.0, recovery=0.0):
3     """带参数化假设的问题2 策略计算。返回 (n_lend, allocated, profit, 结构
   表)。"""
4     turnover_ratio = scorer["turnover_ratio"] if turnover_ratio is None
   else turnover_ratio
5     rate_grid = scorer["rate_grid"]
6     cf = churn_fns_from(scorer, churn_scale)
7     rating_cap = scorer["rating_cap"]
8
9     # 分层阈值: 从 strategy_p2 的层级 PD 均值反推, 再按 tier_shift 平移
10    means = strat.groupby("tier")["PD"].mean()
11    b = [(means["A"] + means["B"]) / 2, (means["B"] + means["C"]) / 2, (
   means["C"] + means["D"]) / 2]
12    b = [x * (1 + tier_shift) for x in b]
13
14    pd_vals = strat["PD"].values
15    tiers = []
16    for p in pd_vals:
17        if p < b[0]:
18            tiers.append("A")
19        elif p < b[1]:
20            tiers.append("B")
21        elif p < b[2]:

```

```

22         tiers.append("C")
23     else:
24         tiers.append("D")
25     tiers = np.array(tiers)
26
27     out = pd.DataFrame({"企业代号": strat["企业代号"], "PD": pd_vals, "tier": tiers})
28     amt = out[["企业代号"]].merge(f2[["企业代号", "out_sum_amt_pos"]], on="企业代号")["out_sum_amt_pos"].values
29     out["rate_opt"] = np.nan
30     out["profit_unit"] = 0.0
31     for i, row in out.iterrows():
32         r, p = optimal_rate_and_profit(row["PD"], row["tier"], cf, rate_grid, recovery)
33         out.loc[i, "rate_opt"] = r if r is not None else np.nan
34         out.loc[i, "profit_unit"] = p
35         cap = np.minimum(out["tier"].map(rating_cap).values, turnover_ratio * amt / 1e4)
36         out["cap_wan"] = np.clip(cap, 0, None)
37         out["amount_wan"] = 0.0
38         order_idx = np.argsort(-out["profit_unit"].values, kind="stable")
39         remaining = total_wan
40         for i in order_idx:
41             if out.loc[i, "profit_unit"] <= 0 or out.loc[i, "cap_wan"] <= 0:
42                 continue
43             take = min(out.loc[i, "cap_wan"], remaining)
44             out.loc[i, "amount_wan"] = take
45             remaining -= take
46             if remaining <= 1e-6:
47                 break
48         out["expected_profit"] = out["amount_wan"] * out["profit_unit"]
49         n_lend = int((out["amount_wan"] > 0).sum())
50         allocated = total_wan - remaining
51         profit = out["expected_profit"].sum()
52         struct = out[out["amount_wan"] > 0].groupby("tier")["amount_wan"].sum().to_dict()
53     return n_lend, allocated, profit, struct

```

Listing 6: run_strategy: 参数化假设下的策略重算（支撑 tornado 图）